

MEMO

Grant Allocation Pipeline - Production Readiness & AI Application

Github link: <https://github.com/LarrineMulunda/one-acre-grant-engine>

This is fundamentally a matching-and-exhaustion problem: Finance needs every eligible transaction matched to a grant, deterministically and idempotently, with accuracy and auditability as the two properties that matter most given money is involved. This memo covers what changed to make Exercise 1's logic production-ready: the code now lives in GitHub with a real change process, data quality is enforced by Soda Core, the platform follows a medallion architecture, Prefect handles orchestration and notifications, and pipeline failures now get AI-assisted triage embedded directly in the alert.

The chosen design, prioritizes determinism, auditability and operational resilience while keeping compute costs efficient. AI is used to augment engineering and finance workflows without compromising the integrity of the financial ledger.

Part 1 - Deployment and Production Readiness

1. Deployment & Release Process

1.1 Version Control & Change Management (GitHub)

- **Storage.** Code lives in a GitHub repository. .gitignore excludes secrets and generated artifacts - nothing sensitive is version controlled.
- **Change process.** Every Pull Request triggers automated pytest. Only successful builds may be merged into main.

1.2 Deployment Strategy / Load Strategy

- **Architecture.** The platform follows an ELT approach with a medallion architecture - Bronze, Silver, and Gold layers.
- **Grants data.** Full load, both Bronze and Silver layer. Grants don't have a reliable watermark column to key an incremental load off, and the table is small enough that a full reload every run is simpler and safer than maintaining incremental logic for negligible benefit.
- **Transaction data.** Incremental load, both Bronze and Silver layer. **TransactionDate** is the watermark column, tracked in an **etl_watermarks** control table. The watermark only advances after a fully successful load - a failed run leaves it untouched, so the next run correctly retries the same window instead of silently skipping data. Writes use MERGE rather than INSERT, which is what makes reruns idempotent.
- **Gold allocation tables.** A hybrid approach, controlled by the accounting period. Closed (past) periods are immutable - their allocation data does not change. The current open period is fully recomputed on every run. This keeps the ledger accurate while minimizing compute, since only the open period's data actually changes run to run. Period state is tracked in an **accounting_periods** control table.

2. Reliability & Resilience

- **Retry policy.** Exponential backoff, up to 3 attempts. Many production failures are transient - network glitches, database contention, API throttling - and resolve themselves on retry without intervention.
- **Idempotency.** Incremental loads use MERGE (ON CONFLICT in SQLite) rather than INSERT, so a retrieved or rerun load cannot create duplicate records.
- **Accurate watermark maintenance.** The watermark only advances after a successful load. If the pipeline fails partway through, the watermark stays where it was, and the next run correctly reprocesses the same window rather than assuming it already succeeded.

- **Rolling window.** Incremental loads use a rolling window and MERGE, which together capture late-arriving data correctly rather than missing it. Accounting periods are closed only after a buffer of X days past the period's end date, this avoid closing a period before late-arriving transactions have landed.
- **Root cause analysis.** After any recovery, the root cause is investigated specifically to prevent recurrence - restoring service and understanding why it failed are treated as two separate steps, not one.
- **Unit tests.** A pytest suite covers the allocation engine's core logic - eligibility, drawdown, tie-break ordering, partial funding and reconciliation. The test is also triggered on merge to main github branch.

3. Monitoring & Alerting

3.1 Pipeline Health Monitoring

- **Alerting.** Success and failure alerts are sent by email via Prefect's flow-state hooks.
- **Duration tracking.** Execution duration is captured at runtime for every task and this can be fed in dashboard for trend visibility - a run that's gradually slowing down is visible before it becomes an outright failure.

3.2 Data Freshness

- **Freshness check.** Soda Core checks whether transaction data is arriving on schedule, flagging a quiet source system before it becomes a bigger problem.

3.3 Infrastructure Monitoring

- **Orchestration-layer visibility.** Prefect's own UI surfaces task-level run history, retry counts, and duration trends. Additionally, infrastructure health can be monitored using cloud monitoring native like Grafana to detect resource exhaustion before it affects pipeline execution.

3.4 Logging

- **Structured, per-task logging.** Every task logs through Prefect's run logger, giving each task's output a distinct, traceable place in the run history rather than one undifferentiated log stream - a failure's log output is scoped to the exact task that produced it.

4. Data Quality

- **Where it runs.** Soda Core enforces data quality gates at both the Bronze and Silver layers, ensuring only accurate, validated data reaches the allocation engine.
- **What it checks.** Duplicate rows, missing rows, invalid values, nulls, unique-key violations, and referential integrity against the grants dimension.
- **Reconciliation.** Reconciliation logic runs on every allocation – This validates $\Sigma \text{ allocated} + \Sigma \text{ unallocated} = \Sigma \text{ Transactions}$ and no negative balances. This is the final, non-negotiable check before any output is trusted.

Part B - AI Application

Deterministic logic governs the core ledger and stays untouched by AI in both applications below. One is built and running today; the second is a proposed extension for a later phase.

5. AI Application in Modern Data Engineering

5.1 AIOps - Initial Triage (built)

- **What it does.** AI root-cause analysis is embedded directly inside failure notifications. The error traceback is passed to the model, which returns step-by-step guidance on how to resolve it - alongside the raw traceback itself, not instead of it.

- **Expansion path.** This could be extended further where AI identifies underlying code issues and improvements, proposing a specific change, and opening a PR -which is reviewed and merged by an engineer, never auto-merged.
- **Governance.** No code change happens without a human in the loop. The AI section notification is explicitly labelled as AI-generated, and the triage message is always delivered alongside the actual failure error
- **Cost.** The call fires only on failure events and uses a lower-cost model tier.
- **Impact.** Reduces mean time to resolution by providing initial diagnosis and remediation steps, surfaces suggestion for code refactoring and improvement.

5.2 AI Insights Embedded in Reporting (proposed)

- **What it does.** Moves reporting from purely descriptive to predictive – predicting which grants will become exhausted before period end and funding gaps based on historical patterns, rather than only reporting what already happened.
- **Insight generated.** Surfaces which grant categories Finance should prioritize getting in order to convert currently-unallocated transactions into allocated ones - reframing the work from “why wasn’t this transaction allocated” to “which grants would most improve our allocation rate right now.”
- **Governance.** Sits on top of normal reporting, not inside the ledger itself, and is explicitly labelled as AI-generated in the dashboard.
- **Cost.** A weekly cadence is sufficient - this is a planning aid, not an operational control, so it doesn't need daily refresh.
- **Impact.** Shifts Finance's effort from investigating individual unallocated transactions after the fact to proactively pursuing the grants that will most improve the period's allocation rate.

6. Other Architectural decision made

- **Vectorised NumPy allocation engine instead of pandas:** Uses numpy array operations and Boolean masking instead of row-by-row loops, reducing memory usage and improving performance.
- **Open and closed period:** Closed periods are immutable to preserve historical financial reports and maintain auditability. Only the current open period is recalculated during each run, with opening grant balances seeded from the most recently closed period. This balances financial correctness with computational efficiency.
- **Control Tables** - Pipeline execution state (watermarks and accounting periods) is maintained in database control tables rather than hardcoded in code. This makes the pipeline configurable, restartable and fully auditable without requiring code changes.
- **Reopen Cascade Function:** Built-in capability to safely reopen one or more previously closed accounting periods when late-arriving transactions, back-dated grants, or data corrections affect historical allocations.

7. Open Engineering Questions

- How should unallocated amounts be treated at the end of an accounting period - carried forward for review, written off, or flagged for a specific Finance action?
- What is the process for adding new grants - Can a grant be added today and start date back dated?